

ROBOT EMBEDDED PROGRAMING

Lab 5: Publisher and Subscriber



Name:

Lance Quan

Tyler Brannon

Percy Sanford

Michael McCorkell

HUMBER POLYTECHNIC

FACULTY OF APPLIED SCIENCES AND TECHNOLOGY

Objective

This lab manual guides you through creating and running a simple ROS 2 publisher and subscriber using Python. You will build a package, add dependencies, create entry points, and run the nodes to observe communication in action.

Step 1: Prepare Your System and Source ROS 2 Jazzy

Open a terminal and run:

Update and upgrade Ubuntu packages

```
sudo apt update && sudo apt upgrade -y
```

Source ROS 2 Jazzy environment (adjust path if installed elsewhere)

```
source /opt/ros/jazzy/setup.bash
```

Create the workspace

```
mkdir -p ~/ros2_ws/src
```

```
cd ~/ros2_ws/
```

```
colcon build
```

This ensures your system is up to date, ROS 2 is on your path, and the workspace structure exists.

Step 2: Create a Package

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python first_package
```

Step 3: Open VS

```
Code .
```

Step 4: Add Dependencies in package.xml

Open the file 'package.xml' inside your 'first_package'folder and add the following dependencies before "export": (Make sure to SAVE)

```
<depend>rclpy</depend>
```

```
<depend>std_msgs</depend>
```

Step 5: Create Publisher Node (demo.py)

Inside the 'first_package/first_package/' folder, create a file named **demo.py** with the following code: (Make sure to SAVE)

```
import rclpy

from rclpy.node import Node

from std_msgs.msg import String


class MinimalPublisher(Node):

    def __init__(self):

        super().__init__('minimal_publisher')

        self.publisher_ = self.create_publisher(String, 'topic', 10)

        self.timer = self.create_timer(0.5, self.timer_callback)

        self.i = 0


    def timer_callback(self):

        msg = String()

        msg.data = f'Hello, ROS 2: {self.i}'

        self.publisher_.publish(msg)

        self.get_logger().info(f'Publishing: "{msg.data}"')

        self.i += 1


def main(args=None):

    rclpy.init(args=args)

    node = MinimalPublisher()

    rclpy.spin(node)

    node.destroy_node()
```

```
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Similarly, Inside the 'first_package/first_package/' folder, create a file named **listener.py** with the following code: (Make sure to SAVE)

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String


class MinimalSubscriber(Node):
    def __init__(self):
        super().__init__('minimal_subscriber')
        self.subscription = self.create_subscription(
            String,
            'topic',
            self.listener_callback,
            10)

    def listener_callback(self, msg):
        self.get_logger().info(f'I heard: "{msg.data}"')

def main(args=None):
    rclpy.init(args=args)
    node = MinimalSubscriber()
```

```
    rclpy.spin(node)

    node.destroy_node()

    rclpy.shutdown()


if __name__ == '__main__':

    main()
```

Step 6: Add Entry Points in setup.py

Open 'setup.py' and add the following under 'entry_points': (Make sure to SAVE)

```
entry_points={

    'console_scripts': [

        'talker = first_package.demo:main',

        'listener = first_package.listener:main',

    ],

},
```

Step 7: Build the Workspace

From the workspace root:

```
cd ~/ros2_ws/
# Source Jazzy again to ensure the build tools see ROS 2
source /opt/ros/jazzy/setup.bash
colcon build
# Source the newly built workspace
source install/setup.bash
```

Step 8: Run the nodes

Open two terminals. In each terminal, always source ROS 2 Jazzy and your workspace before running:

```
source /opt/ros/jazzy/setup.bash

source ~/ros2_ws/install/setup.bash
```

Terminal 1 – Publisher

ros2 run first_package talker

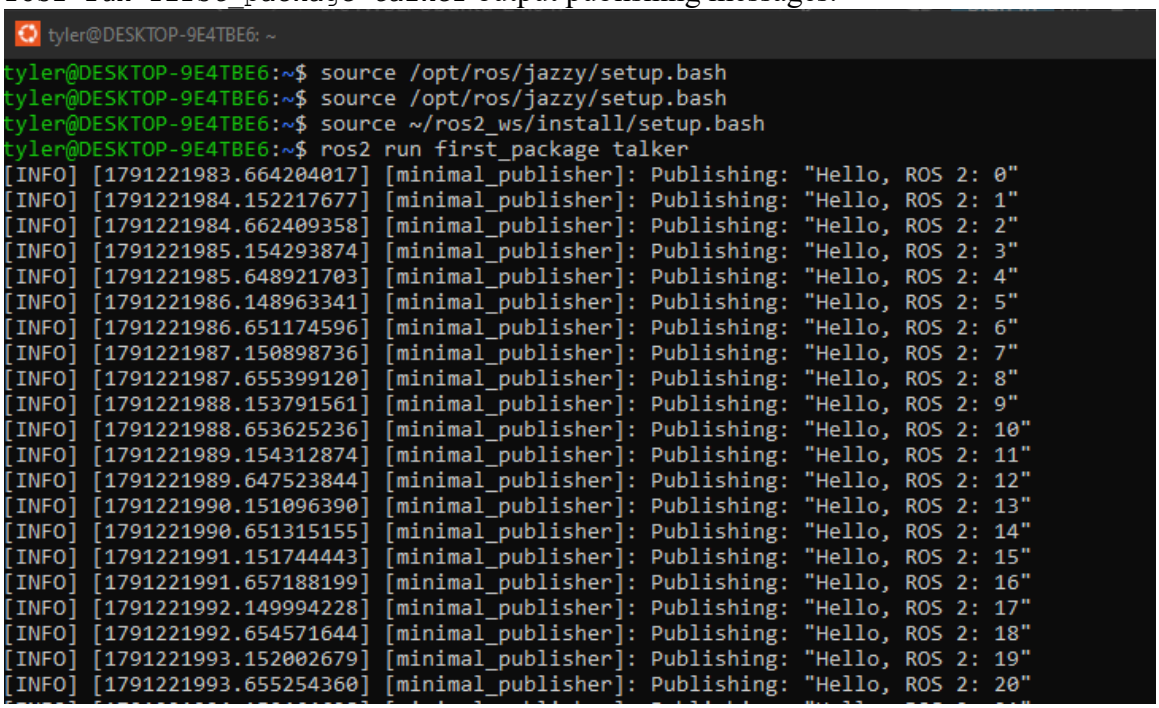
Terminal 2 – Subscriber

ros2 run first_package listener

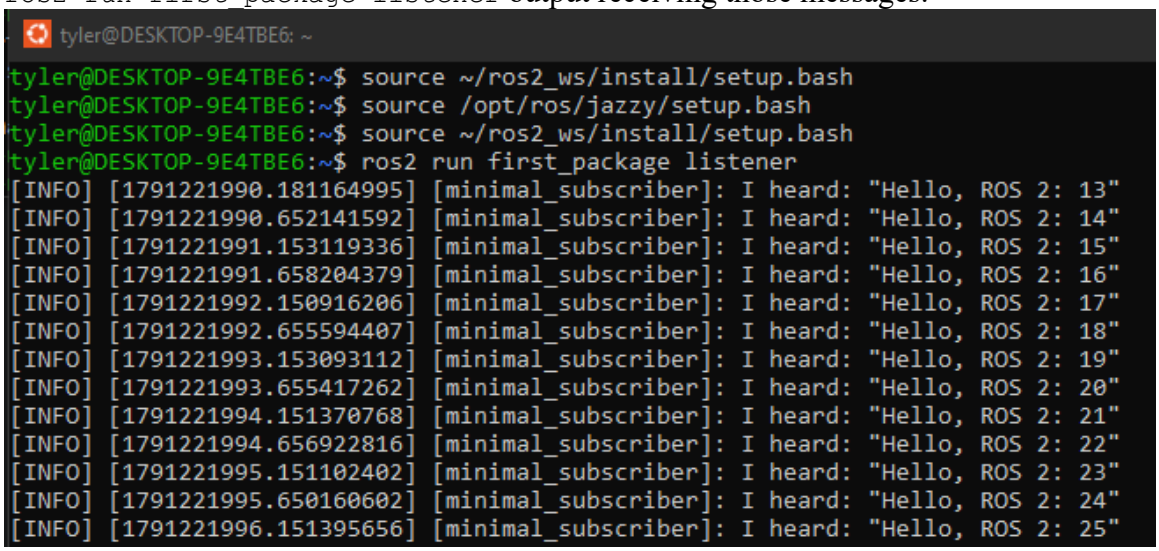
Step 9: Deliverables

1. Screenshots (or a short terminal recording) showing:

- `ros2 run first_package talker` output publishing messages.



- `ros2 run first_package listener` output receiving those messages.



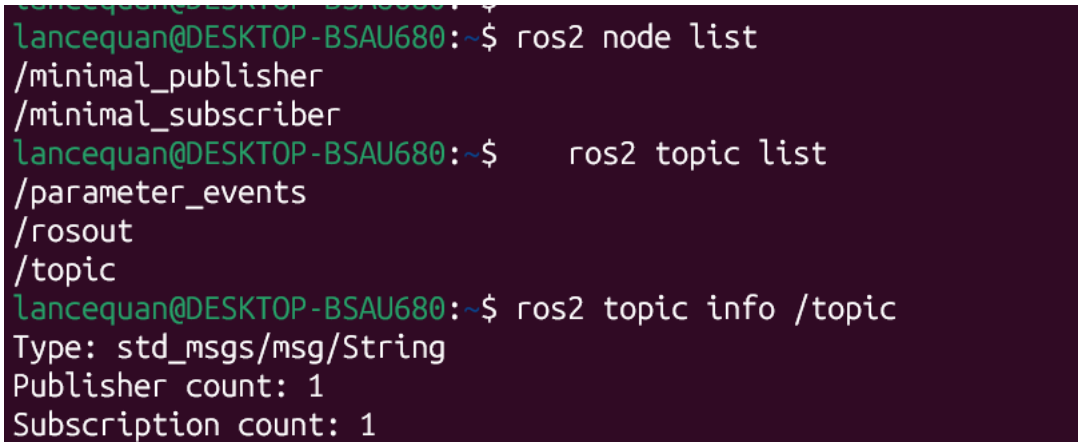
Include the commands used to source the Jazzy environment and the workspace in each terminal.

2. A short paragraph (3-5 sentences) describing what happens between the publisher and subscriber, and what you learned about ROS 2 communication.

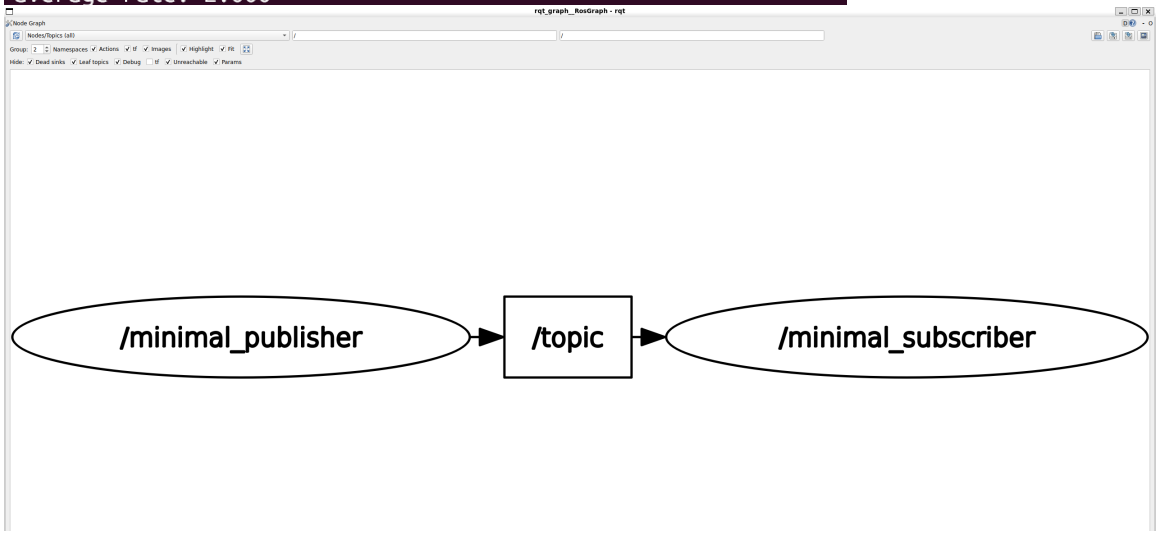
In ROS 2, the publisher sends messages to a topic, while the subscriber receives and processes those messages. When the publisher runs, it continuously publishes data to the topic, the subscriber will receive the data as long as both nodes are connected to the same topic and use the same message type. This allows the two nodes to communicate without directly connecting to each other.

3. **Inspect the running system.** While both nodes are running, open a third terminal (sourced) and run:

```
ros2 node list
ros2 topic list
ros2 topic info /topic
ros2 topic hz /topic
rqt_graph
```



```
lancequan@DESKTOP-BSAU680:~$    ros2 topic hz /topic
average rate: 1.999
        min: 0.499s max: 0.501s std dev: 0.00074s window: 3
average rate: 1.999
        min: 0.499s max: 0.501s std dev: 0.00060s window: 6
average rate: 2.000
        min: 0.499s max: 0.501s std dev: 0.00061s window: 9
average rate: 1.999
        min: 0.499s max: 0.502s std dev: 0.00068s window: 12
average rate: 1.999
        min: 0.498s max: 0.502s std dev: 0.00098s window: 15
average rate: 2.000
        min: 0.497s max: 0.502s std dev: 0.00112s window: 18
average rate: 2.000
        min: 0.497s max: 0.503s std dev: 0.00119s window: 20
average rate: 1.999
        min: 0.497s max: 0.503s std dev: 0.00117s window: 22
average rate: 1.999
        min: 0.496s max: 0.503s std dev: 0.00144s window: 25
average rate: 2.000
        min: 0.496s max: 0.503s std dev: 0.00152s window: 28
average rate: 2.000
        min: 0.496s max: 0.503s std dev: 0.00151s window: 30
average rate: 1.999
        min: 0.496s max: 0.503s std dev: 0.00148s window: 32
average rate: 2.000
        min: 0.496s max: 0.503s std dev: 0.00152s window: 35
average rate: 2.000
```



Include screenshots, then answer:

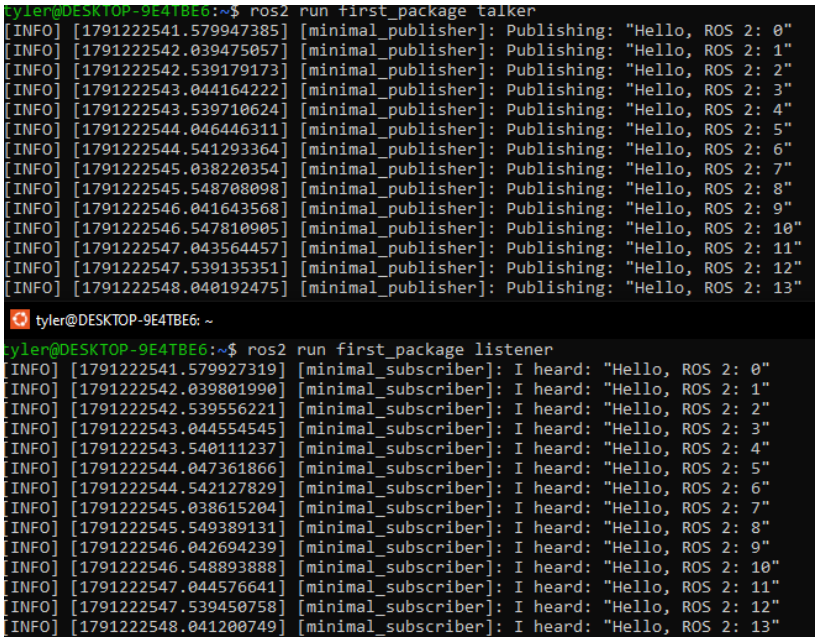
- What message type is used on `/topic`, and how many publishers and subscribers are attached?

  The message type used on /topic is std_msgs/msg/String. When both nodes are running, there is one publisher and one subscriber connected to the topic.

- What publishing rate does `ros2 topic hz` report? Which line of `demo.py` sets this rate?

  The expected publishing rate is approximately 2 Hz, meaning two messages are published every second. This rate is controlled by the line self.timer = self.create_timer(0.5, self.timer_callback) in demo.py, which sets the timer interval to 0.5 seconds.

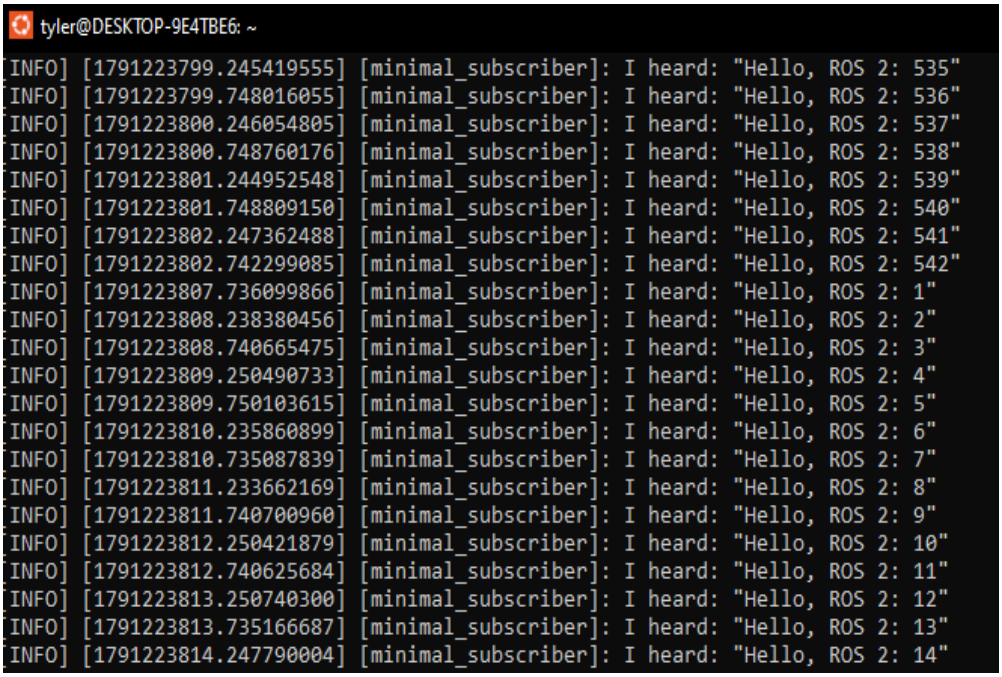
4. **Predict, then test.** For each scenario, write your prediction first, then run it and record what happens:
   - **(a)** Start the listener first, wait 10 seconds, then start the talker. Does the listener receive message 0?



The listener will still receive message 0, the listener will not publish any messages until it receives the message from the talker. Once the talker begins message 0 will be the first result.

This result was correct; the listener waited until the talker began and did not show an output until that happened.

(b) Stop the talker (Ctrl+C) while the listener is still running. What happens to the listener?



The listener will continue to run but will not continue displaying the messages. Once the talker begins again the listener will not continue with the number sequence but will instead reset it back to message 0.

(c) Change the topic name in `listener.py` from `'topic'` to `'chatter'`, rebuild, and run both nodes. What happens, and why? Restore the original name afterward.

When the topic name in listener.py is changed from topic to chatter, the listener will no longer receive messages from the talker. This is because the talker is still publishing messages to topic, while the listener is subscribing to chatter. Since the publisher and subscriber are using different topic names, they cannot communicate. Changing the listener's topic name back to topic allows it to receive messages again.